



# RAG Mini Project RFP

## 1. 프로젝트 Mission

AI 기본법을 잘 모르는 사용자도 질문을 통해 쉽게 내용을 이해할 수 있는 근거 기반 AI 기본법 QA 백엔드 서비스를 구현한다.

이번 프로젝트에서는 법률 문서를 직접 로딩하고, 구조에 맞게 청킹한 뒤 Vector DB에 저장하여 검색 가능한 형태로 만든다.

이후 사용자의 질문과 관련된 법률 조항을 검색하고, Advanced RAG 기법을 적용하여 검색 품질을 개선한 뒤 근거 기반 답변을 생성한다.

| 구분      | 핵심 질문                       | 주요 작업                                                         |
|---------|-----------------------------|---------------------------------------------------------------|
| Offline | 문서를 어떻게 검색 가능한 지식으로 만들 것인가? | Loading → 구조 분석 → Chunking → Embedding → Qdrant 저장            |
| Online  | 질문에 필요한 근거를 어떻게 찾아 답변할 것인가? | Query → Retrieval → 검색 개선 → Reranking → LLM → Answer + Source |

## 프로젝트 목표

이번 프로젝트에서는 다음 4가지를 경험하는 것이 핵심이다.

1. 법률 문서를 RAG에 적합한 형태로 구조화한다.
2. Vector DB에서 관련 문서를 검색한다.
3. Advanced RAG 기술을 적용하여 검색 품질을 개선한다.
4. 검색된 근거를 기반으로 답변을 생성하고 평가한다.

## 프로젝트 구조

프로젝트 템플릿 git url :

[https://github.com/magpia000/gg1th\\_rag\\_mini\\_pjt\\_template.git](https://github.com/magpia000/gg1th_rag_mini_pjt_template.git)

```
rag_minipjt/  
|
```

```

├── data/
│   └── ai_basic_law/
├── rag/
│   ├── __init__.py
│   ├── loader.py
│   ├── chunker.py
│   ├── vectorstore.py
│   ├── retriever.py
│   ├── reranker.py
│   └── pipeline.py
├── app/
│   ├── __init__.py
│   └── main.py
├── eval/
│   ├── golden_set.jsonl
│   └── evaluate.py
├── common/
│   ├── __init__.py
│   ├── ai_model.py
│   ├── config.py
│   └── qdrant.py
├── pyproject.toml
├── .env
└── README.md

```

## 각 모듈의 역할

| 모듈                          | 역할                    |
|-----------------------------|-----------------------|
| <code>loader.py</code>      | AI 기본법 문서 로딩          |
| <code>chunker.py</code>     | 법률 구조 기반 Chunk 생성     |
| <code>vectorstore.py</code> | Embedding 및 Qdrant 저장 |
| <code>retriever.py</code>   | 관련 문서 검색              |
| <code>reranker.py</code>    | 검색 결과 재정렬             |
| <code>pipeline.py</code>    | 전체 RAG Pipeline 연결    |
| <code>app/main.py</code>    | FastAPI 서비스           |
| <code>eval/</code>          | Golden Test Set 기반 평가 |

## 사용할 데이터

인공지능 발전과 신뢰 기반 조성 등에 관한 기본법 (약칭: 인공지능기본법)

2026년 7월 개정된 자료

## AI 기본법(2026년 7월 개정 법령).

인공지능 발전과 신뢰 기반 조성 등에 관한 기본법 ( 약칭: 인공지능기본법 )

[시행 2026. 7. 21.] [법률 제21311호, 2026. 1. 20., 일부개정]

The screenshot shows the official Korean Legal Information System (KOLIS) website. The search bar at the top contains '인공지능' (AI). The main content area displays the '인공지능 발전과 신뢰 기반 조성 등에 관한 기본법' (Artificial Intelligence Development and Trust Foundation Establishment Basic Act). The left sidebar lists various legal documents, with the current document highlighted. The main text area shows the '제1장 총칙' (General Provisions) section, including the purpose of the law and the definition of 'AI'.

## 2. 팀에서 구현할 핵심 부분

이번 미니프로젝트에서는 모든 기능을 처음부터 구현하는 것이 아니라, 강사가 제공한 기본 구조를 활용하여 **RAG의 핵심 흐름을 직접 구현하고 개선하는 것에 집중한다.**

문서 로딩 → 구조 기반 청킹 → Vector DB 저장 → 검색 → 개선 → 답변 → 평가

### 1) 문서 로딩·구조화·Vector DB 저장

AI 기본법 문서를 로딩하고, 법률 구조를 고려하여 검색에 적합한 Chunk로 변환한 뒤 Qdrant에 저장한다.

AI 기본법 문서  
↓  
Document Loading  
↓  
법률 구조 분석  
↓  
Chunking  
↓  
Embedding

↓  
Qdrant Vector DB 저장

핵심 구현 내용은 다음과 같다.

- Docling을 이용한 법률 문서 로딩
- 장·절·조·항 등 법률 구조 확인
- 구조 기반 Chunk 생성
- Metadata 구성
- Embedding 생성
- Qdrant Vector DB 저장

관련 모듈은 다음과 같다.

```
rag/loader.py  
rag/chunker.py  
rag/vectorstore.py
```

## 2) Retrieval

사용자의 질문과 관련된 법률 조항을 검색한다.

기본 Dense Retrieval부터 시작하고, 필요하면 검색 품질을 높이기 위한 Advanced RAG 기법을 적용한다.

```
사용자 질문  
↓  
Query Embedding  
↓  
Qdrant 검색  
↓  
Top-K 문서  
↓  
검색 품질 개선  
↓  
Reranking
```

적용 가능한 방법은 다음과 같다.

- Multi Query
- Hybrid Search

- BM25
- RRF
- Metadata Filtering
- Reranking

관련 모듈은 다음과 같다.

```
rag/retriever.py
rag/reranker.py
```

중요한 것은 여러 기법을 많이 적용하는 것이 아니라, **어떤 방법이 실제 검색 결과를 개선했는지 확인하는 것**이다.

### 3) Grounded Answer

검색된 법률 문서를 Context로 사용하여 LLM이 근거 기반 답변을 생성하도록 구현한다.

```
사용자 질문
↓
Retrieval 결과
↓
Context 구성
↓
LLM
↓
Answer + Source
```

답변에는 가능한 경우 다음 내용을 포함한다.

답변

근거

- 관련 조문
- 관련 항
- 검색된 법률 내용

관련 모듈은 다음과 같다.

```
rag/pipeline.py
app/main.py
```

핵심은 LLM이 자신의 일반 지식만으로 답변하지 않고, 검색된 법률 문서를 근거로 답변하도록 만드는 것이다.

## 4) Evaluation

Golden Test Set을 이용하여 검색과 답변 품질을 평가한다.

```
Golden Question
↓
Retrieval
↓
정답 문서 검색 여부 확인
↓
LLM Answer
↓
근거와 답변 품질 평가
```

Retrieval은 다음과 같은 지표를 활용할 수 있다.

- Hit@K
- Recall@K
- MRR

Answer는 다음 기준으로 평가한다.

- 정답성
- 관련성
- 근거 충실성
- Hallucination 여부

관련 파일은 다음과 같다.

```
eval/golden_set.jsonl
eval/evaluate.py
```

이번 프로젝트의 핵심 흐름은 다음과 같다.

1. 문서를 검색 가능한 지식으로 만든다.  
↓
2. 필요한 근거를 잘 찾는다.  
↓

3. 근거를 기반으로 답변한다.

↓

4. 실제로 잘 동작하는지 평가한다.

**Document Processing → Retrieval → Grounded Answer → Evaluation**

의 전체 RAG 파이프라인을 이해하고 구현하는 것이 이번 팀 미니프로젝트의 핵심이다.

## API 결과물

최종적으로 다음 API가 동작하면 된다.

POST /ask

입력

```
{
  "question": "고영향 인공지능이란 무엇인가요?"
}
```

출력

```
{
  "answer": "고영향 인공지능은 ...",
  "sources": [
    {
      "article": "제○조",
      "content": "..."
    }
  ]
}
```

## 이번 프로젝트에서 중요한 것

이번 프로젝트는 코드를 많이 만드는 프로젝트가 아니다.

다음 질문에 답할 수 있는 시스템을 만드는 것이 목표이다.

이 답변은 어떤 문서를 근거로 만들어졌는가?

｜ 필요한 법률 조항을 제대로 검색했는가?

｜ 검색 품질을 어떻게 개선할 수 있는가?

｜ 개선된 결과가 실제로 더 좋아졌는가?

즉,

**문서 구조화 → 검색 → 검색 개선 → 근거 기반 답변 → 평가**

이 전체 RAG 흐름을 직접 경험하는 것이 이번 미니프로젝트의 핵심이다.